

CardioCare

End-to-End ML System for Heart Disease Prediction

기계학습 기말 프로젝트 보고서 | 2026-06-12 | 최한결

1. 문제 정의와 사용 목적

CardioCare는 환자의 나이, 혈압, 콜레스테롤 등 13가지 임상 수치를 입력 받아 심장병 가능성을 수치(확률)로 제시하는 시스템입니다. 단독 결정 도구가 아닌 이유는, 모델이 과거 303명의 데이터 패턴을 학습한 것에 불과하기 때문에 개별 환자의 복약 이력, 가족력, 최신 검사 맥락 같은 정보는 반영하지 못합니다. 따라서 이 시스템의 출력은 심장 전문의가 놓칠 수 있는 위험 신호에 주의할 기율이도록 돕는 참고 지표이며, 진단과 치료 결정의 최종 책임은 반드시 의료진에게 있습니다.

1.1 데이터셋

데이터는 UCI Machine Learning Repository의 Heart Disease — Cleveland Clinic Foundation 서브셋을 ucimlrepo API로 로드했습니다. 규모는 303행 × 13특성이며, 원본의 다중 클래스 타깃 num을 num > 0이면 1(심장병), 0이면 정상으로 이진화했습니다. 클래스 분포는 정상 164건, 심장병 139건으로 경미한 불균형입니다. 결측치는 ca(4개)와 thal(2개)에만 있으며, ucimlrepo가 처음부터 NaN으로 제공합니다.

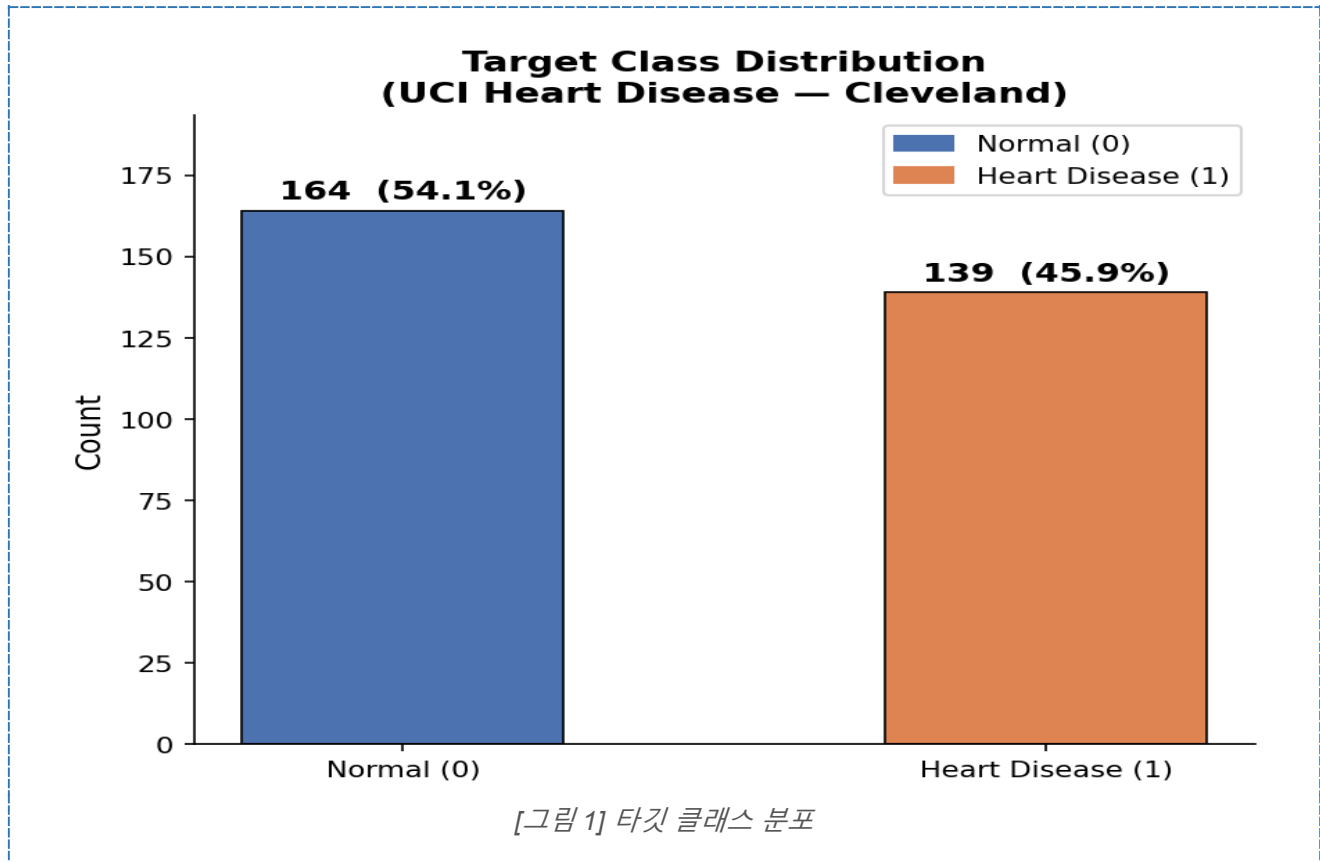
1.2 시스템 구성 흐름

ucimlrepo로 Cleveland 데이터를 로드한 뒤, EDA 노트북에서 분포·결측·이상치를 탐색합니다. 탐색 결과를 바탕으로 sklearn Pipeline 안에 중앙값 대체 → IQR 클리핑 → StandardScaler → SelectFromModel 순서로 전처리와 특성 선택을 캡슐화하고, 4개 모델(LR·SVC·RF·KNN)을 MLflow로 추적하며 학습합니다. 테스트셋 평가와 RF 하이퍼파라미터 튜닝을 거쳐 최종 모델을 models/best_model.pkl로 저장하고, src/inference.py와 Dockerfile로 패키징해 Docker 컨테이너로 배포합니다. GitHub Actions CI가 push마다 단위 테스트 4개를 자동 실행하고, src/monitor.py가 KS 검정과 슬라이딩 윈도우로 분포·성능 드리프트를 상시 감시합니다.

2. EDA 핵심 결과

2.1 타깃 분포

노트북에서 확인했을 때 정상 164명, 심장병 139명으로 비율이 54:46이었는데, 이 정도면 심한 불균형이라고 부르기 어렵습니다. 그래도 balanced accuracy와 recall을 주 지표로 선택한 건 불균형 때문이 아니라, 심장병 환자를 정상으로 잘못 분류했을 때의 임상적 결과(치료 지연, 합병증)가 반대 오류보다 훨씬 크기 때문입니다. 단순 accuracy는 이런 오류 비대칭을 전혀 반영하지 못합니다.



2.2 결측치

ca에 4개, thal에 2개, 합쳐서 6개로 전체 데이터의 2%도 안 됩니다. 이 6행을 삭제할 수도 있었지만, 303행짜리 소규모 데이터에서 6행을 날리면 학습 데이터 손실이 체감상 아깝고 통계적으로도 손해입니다. 그래서 ca는 순서형이니 중앙값으로, thal은 범주형이니 최빈값으로 대체하는 쪽을 택했습니다.

2.3 이상치

먼저 chol(콜레스테롤) 변수의 분포를 확인해 본 결과, 최솟값은 126으로 하단은 정상 범위였으나, 상단으로는 371을 넘어 최고 564까지 이어지는 긴 오른쪽 꼬리 형태를 띠고 있었습니다. oldpeak

변수 역시 0 근처에 데이터가 밀집되어 우측으로 크게 치우친 분포를 보였습니다. 이처럼 두 변수 모두 정규분포를 가정하기 어려워 z-score 방식은 적합하지 않았고, 전체 데이터가 303행에 불과해 행을 삭제할 경우 데이터 손실이 너무 크다고 판단했습니다. 따라서 상단 펜스($Q3 + 1.5 \times IQR$)를 기준으로 그 이상의 값만 경계값으로 대체해 주는 IQR 클리핑 방식을 채택했습니다. 하단은 이미 정상 범위 내에 있었으므로, 실제로는 데이터 손실 없이 상단의 극단값만 효과적으로 제어할 수 있었습니다.

2.4 특성과 타깃의 관계

노트북의 클래스별 KDE 그래프에서 thalach(최대 심박수)가 심장병 환자 쪽에서 전반적으로 낮게 분포하는 게 뚜렷하게 보였습니다. oldpeak도 심장병 환자에서 값이 더 퍼져 있고 높은 쪽에 분포가 더 있었습니다. 이 두 특성은 클래스 구분력이 있어 보였고, 이후 SelectFromModel 결과에서도 선택된 특성 목록에 포함됐습니다.

3. EDA 근거 전처리 결정

3.1 파이프라인 구성

이러한 EDA 결과를 바탕으로, 전처리 파이프라인은 ‘중앙값 대치 → IQR 클리핑 → StandardScaler’ 순서로 설계했습니다. 우선 이상치에 강건한 중앙값으로 결측치를 대치한 후, 앞서 말씀드린 클리핑을 통해 데이터를 보존하면서 극단값만 경계로 눌러주었습니다. 마지막으로 StandardScaler를 적용하여, 향후 사용될 SVC나 로지스틱 회귀와 같이 스케일에 민감한 모델에서 발생할 수 있는 성능 왜곡을 사전에 차단했습니다.

3.2 기초 정제 및 방어적 데이터 처리

- **결측/중복 제어:** load_raw_data() 단계에서 모든 값이 NaN 인 컬럼과 중복 행을 선제적으로 제거했습니다.
- Cleveland에는 실제로 완전 결측 컬럼이 없어서 이 코드가 동작하지는 않습니다. 다만 Hungarian-Switzerland 서브셋처럼 데이터 출처가 달라질 경우를 대비해 넣어둔 코드입니다.

3.3 특성 선택 및 EDA 결과의 검증

- **선택 기준:** SelectFromModel 과 Random Forest 를 결합하여, 학습 데이터 기준 Feature Importance 의 중앙값(median) 이상인 특성만 선별했습니다.
- **선택 결과 (총 13 개):** 연속형 — age, trestbps, chol, thalach, oldpeak, ca / 범주형(OHE) — cp=4, exang=0, exang=1, slope=1, slope=2, thal=3, thal=7.
- **분석적 의의:** 전처리 이전 EDA(2.4) 단계에서 심장병 유무에 따라 가장 뚜렷한 분포 차이를 보였던 thalach 와 oldpeak 변수가 실제로 주요 특성으로 채택되었습니다. 이는 탐색적 데이터 분석의 시각적 인사이트가 모델의 수학적 중요도 평가와 일치함을 교차 검증한 결과입니다.

3.4 Pipeline 캡슐화를 통한 데이터 누수 차단

데이터 누수 방지의 핵심 설계는 모든 데이터 변환 및 선택 과정을 Scikit-learn Pipeline 내부에 완전히 캡슐화한 것입니다.

- 파이프라인을 `X_train`에만 `.fit()`하도록 만들었기 때문에, 중앙값(Imputer)·Q1/Q3 경계(IQRClipper)·평균과 분산·특성 중요도 같은 모든 학습 통계가 학습 데이터만 보고 결정됩니다. 테스트셋은 `.transform()`만 적용받으므로 그 정보가 학습에 새어 들지 않습니다.
- 교차 검증에서도 마찬가지입니다. `cross_validate` 와 `RandomizedSearchCV` 는 fold 마다 파이프라인의 `.fit()`을 새로 호출하기 때문에, 해당 fold 의 검증 데이터가 학습 기준에 영향을 주지 않습니다.
- 만약 "왜 누수가 없나요?"라는 질문의 답은 학습되는 모든 변환기가 Pipeline 안에 있고, Pipeline 은 항상 `X_train`에만 fit 하기 때문입니다. 따라서 검증용 Fold 의 정보가 학습용 Fold 의 전처리 기준에 유입되는 Data Leakage 를 원천적으로 방지했습니다. 테스트셋은 최종 평가 시 오직 `.transform()` 및 예측 단계에만 사용됩니다.

4. 모델링, 실험 추적 (MLflow), 모델 선택

4.1 MLflow 실험 추적

`src/train.py`는 모델마다 MLflow run을 생성하고 아래 항목을 기록합니다.

- **태그:** 모델명, 데이터셋 이름, 전처리 전략

- **파라미터:** test_size=0.2, random_seed=42, selector=SelectFromModel(RF, threshold=median), 선택된 특성 수(n_selected_feats)
- **지표:** balanced_accuracy, recall, precision, f1, roc_auc (테스트셋 기준)
- **아티팩트:** 혼동 행렬 PNG, classification report TXT, 선택된 특성 목록 JSON, 모델 pkl

4.2 테스트셋 성능 (80/20 단일 분할, test n=61)

모델	BalAcc	Recall	Precision	F1	AUC	FN	FP
LogisticRegression	0.9064	0.9643	0.8438	0.9000	0.9632	1	5
SVC	0.9064	0.9643	0.8438	0.9000	0.9578	1	5
RandomForest	0.9215	0.9643	0.8710	0.9153	0.9426	1	4
KNN	0.8858	0.8929	0.8621	0.8772	0.9426	3	4

80/20 분할(train 242 / test 61), stratify=target으로 클래스 비율 유지, random_state=42로 재현성 고정했습니다.

BalAcc와 Recall 최댓값 기준으로 RandomForest가 자동 선택되어 models/best_model.pkl로 저장됩니다. LR·SVC와 FN은 동률이지만 FP가 1건 적고 BalAcc가 높았습니다.

4.3 5-Fold Stratified CV (학습셋 n=242 기준)

모델	BalAcc (mean±std)	Recall	Precision	F1
LogisticRegression	0.8321 ± 0.0325	0.8103	0.8400	0.8196
SVC	0.7906 ± 0.0156	0.7652	0.7869	0.7724
KNN	0.7873 ± 0.0224	0.7202	0.8151	0.7624
RandomForest	0.7615 ± 0.0198	0.7375	0.7532	0.7408

CV에서는 LR이 1위, RF가 꼴찌입니다. 이 불일치가 4.5에서 다루는 핵심 쟁점입니다. (roc_auc는 CV scoring에 포함하지 않았으며, AUC는 테스트셋 단일 평가인 4.2 표에서만 보고합니다.)

4.4 RF 하이퍼파라미터 튜닝 (RandomizedSearchCV, n_iter=20, cv=5)

기본 RF의 CV BalAcc(0.7615)가 낮아 RandomizedSearchCV로 하이퍼파라미터를 탐색했습니다.

파라미터	최적값
n_estimators	238
max_depth	15

파라미터	최적값
max_features	'sqrt'
min_samples_split	9
min_samples_leaf	3

튜닝 결과 CV BalAcc는 0.7615 → 0.8114로 향상됐습니다. 그러나 테스트셋에서 튜닝 RF의 BalAcc는 0.9064로, 기본(무튜닝) RF의 0.9215보다 낮았습니다. train.py는 테스트 BalAcc와 Recall의 최댓값으로 최종 모델을 선택하므로, 배포된 models/best_model.pkl은 기본 RF입니다. 이 결과에 대한 해석은 4.5에서 다룹니다.

4.5 최종 모델 선택 — CV 와 테스트 결과의 불일치

(1) 결과 불일치 인정. 4.3의 5-fold CV 표와 4.2의 테스트 결과 표가 서로 다른 순위를 보입니다. CV에서는 RandomForest가 balanced_accuracy 0.7615로 4모델 중 가장 낮고 LogisticRegression이 0.8321로 1위인데, 테스트셋에서는 반대로 RF가 0.9215로 1위입니다. 이 불일치가 존재한다는 점을 먼저 인정합니다.

(2) 원인 분석. 테스트셋이 61개로 매우 작아서 단일 분할 결과는 분산이 높습니다. RF의 경우 CV와 테스트 간 격차가 16%p인데, 이는 이번 80/20 분할이 우연히 RF에 유리한 구성이었다는 신호입니다. 반면 LogisticRegression은 격차가 7%p로 분할에 덜 민감합니다. 일반적으로 5-fold CV가 단일 분할보다 일반화 성능의 더 신뢰할 만한 추정치입니다.

(3) RF 선택의 타이브레이커. 4개 모델 간의 성능 차이가 미미하고 테스트셋의 규모가 한정적인 상황에서, 단일 성능 점수만으로 특정 모델의 절대적 우위를 단정하기는 어렵습니다. 그럼에도 최종 배포 모델로 랜덤 포레스트(RF)를 채택한 기준은 다음과 같습니다.

1. **테스트셋 기준 최고 성능:** 별도로 구성된 테스트셋 평가 결과, Balanced Accuracy 지표에서 가장 높은 수치를 기록했습니다(이 차이는 거짓양성 1 건에서 비롯된 매우 작은 차이입니다).
2. **재현율(Recall) 동률 내 우선순위:** FN 최소화 및 Recall 지표에서 로지스틱 회귀(LR)와 함께 가장 우수한 성능을 보였으며, 그 동률 조건 안에서 Balanced Accuracy가 더 높은 RF에 우선순위를 두었습니다.

3. **거짓양성(FP) 비교:** Recall 과 FN 이 동률인 상황에서 RF 의 FP 는 4 건, LR 의 FP 는 5 건으로, RF 가 정상 환자를 심장병으로 잘못 분류한 건수가 1 건 적었습니다. FN 보다 임상 비용이 낮기는 하지만, 동률 조건 안에서는 FP 도 줄이는 모델이 더 나은 선택이라고 판단했습니다.

하지만, 이는 로지스틱 회귀(LR)의 우수한 교차 검증(CV) 성능과 안정성을 배제하는 것은 아닙니다. LR을 선택했다라도 충분히 타당한 접근이었을 것이며, 향후 데이터가 추가 확보될 경우 본 판단에 대한 재평가가 필요함을 인지하고 있습니다.

(4) **투명성.** CV 결과와 테스트 결과를 모두 보고서에 병기하고, 두 지표가 다른 순위를 내놓는다는 점을 숨기지 않았습니다. 원칙적으로는 CV로 모델을 선택하고 테스트셋은 마지막에 단 한 번만 보고하는 것이 정석이며, 이번 구현에서는 테스트 점수로 비교해 선택했기 때문에 그 테스트 점수가 약간 낙관적으로 편향돼 있을 수 있다는 점도 인지하고 있습니다.

5. 테스트, 패키징, CI/CD

5.1 단위 테스트 (tests/test_pipeline.py)

인터넷·MLflow 없이 합성 데이터만으로 돌아가도록 설계했습니다. 실행 결과: Ran 4 tests in 0.894s OK.

테스트	검증 내용
test_prediction_output_shape	predict() 반환 배열의 shape이 입력 행 수와 일치하는지
test_predict_proba_range_and_sum	predict_proba()의 각 행 합이 1.0이고 값이 [0,1] 범위인지
test_clinical_feature_ranges	임상 범위를 벗어난 입력(chol=700, age=-5, oldpeak=-1 등)이 ValueError를 발생시키는지
test_pipeline_determinism	동일 입력·동일 시드에서 두 번 fit해도 결과가 동일한지

5.2 Docker 패키징

python:3.10-slim 베이스 이미지에 의존성 레이어(requirements)와 소스 레이어(src/)를 분리해 빌드 캐시 효율을 높였습니다. 비루트 사용자(appuser)를 생성해 컨테이너 보안을 강화했으며, 개발 중 logs/ 디렉터리가 root 소유로 생성되어 appuser가 쓰지 못하는 권한 오류(PermissionError: [Errno 13])가 발생했습니다. useradd와 mkdir -p logs && chown -R appuser:appuser logs를 같은

RUN 레이어 안에 묶어 해결했습니다. 기본 실행 시 data/sample_batch.csv로 추론을 수행하고, 커스텀 입력은 -v 볼륨 마운트로 주입할 수 있습니다.

5.3 GitHub Actions CI

.github/workflows/ci.yml은 push(전 브랜치)와 main/master PR에서 트리거됩니다. Python 3.10 환경을 세팅하고 pip install -r requirements.txt → python -m py_compile로 3개 소스 파일 문법 검사 → python -m unittest discover -s tests -v 순서로 실행합니다. 단위 테스트가 합성 데이터를 사용하기 때문에 CI 환경에서 인터넷 연결이나 사전 학습 모델 없이도 4개 테스트가 모두 통과합니다.

5.4 피처 스토어 & 모델 레지스트리

피처 스토어에 등록할 피처: chol (혈청 콜레스테롤)

chol은 식이 변화, 약물 복용, 계절에 따라 수치가 달라지는 시간 가변 특성입니다. 학습 시점의 chol 분포와 실제 운영 시점의 분포가 달라지면 모델이 엉뚱한 기준으로 판단하는 학습-서빙 스큐(training-serving skew)가 생깁니다. Feast 같은 피처 스토어에 chol을 등록해 두면, 학습과 추론 모두 동일한 버전·시점의 값을 가져와 쓸 수 있어 이 스큐를 방지할 수 있습니다. 또한 전자의무기록(EMR) 시스템과 연동해 최신 검사값을 자동으로 피처 스토어에 적재하면, inference.py가 별도 전처리 없이 일관된 입력을 받을 수 있습니다.

모델 레지스트리에 기록할 메타데이터: balanced_accuracy + recall

단순 accuracy는 클래스 불균형 상황에서 다수 클래스에 편향될 수 있고, 의료 시스템에서는 심장병 환자를 놓치는 FN의 임상 비용이 훨씬 크기 때문에 recall이 핵심 지표입니다. MLflow Model Registry에 balanced_accuracy와 recall을 버전마다 기록해 두면, 차기 모델을 등록할 때 이 두 지표가 이전 버전보다 낮으면 자동으로 배포를 차단하는 게이트 정책을 설정할 수 있습니다. 이는 성능이 나빠진 모델이 실수로 운영 환경에 배포되는 것을 방지하는 안전장치입니다. 현재 구현에서는 train.py가 MLflow run마다 두 지표를 기록하고 있어, 레지스트리 연동의 기반이 갖춰진 상태입니다.

6. 드리프트 탐지 & 재학습 전략

6.1 분포 이동 시뮬레이션

- chol: 평균 +30 mg/dL, 표준편차 $\times 1.5$ (식이 변화·스크리닝 인구 변화)
- trestbps: 평균 +15 mmHg, 표준편차 $\times 1.3$ (고령화·고혈압 유병률 증가)
- oldpeak: 표준편차 $\times 1.4$ (운동 강도 프로토콜 변경)

6.2 KS 검정 결과 (scipy.stats.ks_2samp, $\alpha=0.05$)

특성	KS 통계량	p-value	상태
age	0.0876	0.8124	OK
trestbps	0.4638	<0.001	DRIFT
chol	0.1510	0.1916	OK
thalach	0.0706	0.9517	OK
oldpeak	0.4262	<0.001	DRIFT

6.3 KS 민감도의 한계 — chol 사례

실험에서 chol을 +30 이동시켰음에도 KS p값이 0.19로 드리프트를 감지하지 못했습니다. chol은 자연 분산 자체가 크기 때문에(표준편차 약 52 mg/dl), +30 이동이 분포 전체의 변화로 보이지 않고 기존 분산 범위 안에 흡수된 것입니다. 이는 분산이 큰 특성일수록 같은 절대 이동량에 대해 KS 민감도가 낮다는 점을 보여줍니다. 따라서 KS 단독으로는 부족하고, 성능 기반 트리거를 병행하는 이유가 여기에 있습니다.

6.4 성능 비교 (원본 vs 드리프트 테스트셋)

지표	원본 테스트셋	드리프트 테스트셋	변화 (Δ)
Balanced Accuracy	0.9215	0.8885	-0.0330
Recall	0.9643	0.9286	-0.0357
Precision	0.8710	0.8387	-0.0323
F1-Score	0.9153	0.8814	-0.0339

6.5 재학습 트리거 정책

재학습 조건은 세 가지를 OR로 연결했습니다. 첫째, KS 검정에서 $p < 0.05$ 인 특성이 2개 이상 동시에 검출될 때입니다. 특성 하나만 드리프트되면 모델이 나머지 특성으로 보완할 수 있지만, 두 개

이상이면 입력 분포 자체가 달라졌다고 판단할 수 있습니다. 둘째, 슬라이딩 윈도우 `balanced accuracy`가 0.85 아래로 떨어질 때입니다. 분포 통계가 바뀌지 않아도 예측 성능이 실제로 저하되고 있다는 신호이므로, KS를 통과하더라도 이 조건으로 잡을 수 있습니다. 셋째, 위 두 조건에 걸리지 않더라도 3개월마다 정기 재학습을 수행합니다. 임상 환경에서는 측정 장비 교체, 환자군 구성 변화, 진단 기준 개정 같은 점진적 변화가 통계적으로 감지되기 전에 누적될 수 있기 때문입니다.

6.6 Human-in-the-loop 지점

자동 재학습이 완료되더라도 바로 배포하지 않습니다. 재학습 직후 임상 전문가가 세 가지를 검토합니다. 첫째, FN(심장병을 정상으로 잘못 분류한 건수) 비율이 이전 모델보다 증가하지 않았는지 확인합니다. 정확도가 올라도 FN이 늘면 임상적으로 더 위험한 모델입니다. 둘째, 재학습에 사용된 데이터가 실제 환자군을 대표하는지 검토합니다. 특정 기간·시설·연령대 환자만 집중 수집됐다면 편향된 모델이 될 수 있습니다. 셋째, 성별·연령·인종 등 하위 집단별로 성능 격차가 새로 생기거나 벌어지지 않았는지 확인합니다. 이 세 항목을 통과한 모델만 교체가 승인됩니다.

6.7 폭주 피드백 루프 위험과 안전장치

자동 재학습 파이프라인에서 가장 큰 위험은 폭주 피드백 루프입니다. 드리프트된 데이터나 모델이 잘못 예측한 결과가 그대로 재학습 데이터로 쌓이면, 모델은 자신의 오류 패턴을 반복 학습하게 됩니다. 초기에는 작은 편향이었던 것이 재학습을 거듭할수록 증폭되어, 결국 특정 환자군을 체계적으로 놓치는 방향으로 수렴할 수 있습니다. 의료 시스템에서 이 현상은 조용히 진행되다가 나중에 발견되는 경우가 많습니다.

이를 막기 위해 두 가지 안전장치를 두었습니다. 첫째, 재학습에는 임상 전문가가 직접 검토·승인한 레이블만 사용합니다. 모델 예측값이나 미검증 자동 레이블은 학습 데이터에 포함하지 않습니다. 둘째, 6.6의 Human-in-the-loop 검토를 재학습마다 의무화해, 자동화 루프 안에 반드시 사람이 끼어드는 구조를 만들었습니다. 자동 재학습은 속도를 위한 것이고, 배포 여부의 판단은 사람이 내리는 것이 원칙입니다.

7. 서빙 전략 — MaaS vs. On-Device

7.1 배포 방식 비교

비교 항목	클라우드 API (MaaS)	로컬 배포 (On-Device)
추론 속도(지연시간)	병원 인터넷 상태에 따라 예측값을 받아오는 데 지연이 생길 수 있음	병원 내부 컴퓨터에서 직접 추론하므로 네트워크 지연 없이 항상 빠른 결과
환자 개인정보(PHI)	민감한 환자 데이터가 외부 서버로 나가야 해 유출 위험·법적 규제(HIPAA) 부담이 큼	데이터가 병원 내부망을 벗어나지 않아 개인정보 보호 측면에서 매우 안전
모델 업데이트	중앙 서버에서 한 번 교체하면 모든 병원에 즉시 적용되어 관리가 쉬움	병원마다 새 버전을 개별 설치해야 해 관리가 번거로움

7.2 CardioCare 의 최종 선택: On-Device

위의 장단점을 종합해 보았을 때, CardioCare 모델은 On-Device(병원 내부 배포) 방식으로 운영하는 것이 가장 적합하다고 판단했습니다. 그 이유는 다음과 같습니다.

- 민감한 의료 데이터의 안전성:** 병원 데이터는 진단명·심전도 수치 등이 포함된 최고 수준의 민감 정보입니다. 배포 편의를 위해 이를 외부로 전송하는 것은 환자의 신뢰를 잃을 수 있는 위험한 선택이며, 병원 내부에서만 처리해 유출 가능성을 원천 차단하는 것이 우선입니다.
- 응급 상황에서의 안정적 추론 속도:** 흉통·호흡 곤란 환자의 초기 평가에서는 1 초의 지연도 진료 흐름에 방해가 될 수 있습니다. 외부 인터넷 상태에 의존하기보다 로컬에서 항상 일정하고 빠르게 결과를 내는 것이 의료 현장에 더 알맞습니다.
- 감당 가능한 업데이트 주기:** 로컬 배포의 단점은 업데이트의 번거로움이지만, 6 장에서 설계한 대로 의료 모델은 최소 3 개월 단위로 Human-in-the-loop 검토를 거쳐 재학습합니다. 따라서 잦은 배포가 필요하지 않아 이 단점은 충분히 감당할 수 있습니다.

7.3 도입 한계점 및 대안

다만 이 전략에도 한계는 있습니다. 자체 IT 서버 장비나 관리 인력이 부족한 소규모 의원의 경우 온디바이스 구축 비용이 너무 클 수 있습니다. 이 경우에는 데이터 처리 위치를 철저히 병원 소유로 통제하는 프라이빗 클라우드 형태를 빌려 쓰는 방식이 더 현실적인 대안이 될 수 있습니다.

8. 한계, 윤리적 고려, 향후 과제

8.1 데이터 한계

이 프로젝트에서 사용한 데이터는 303행으로, 머신러닝 모델을 학습하기에 넉넉한 규모라고 보기

어렵습니다. 특히 테스트셋이 61개밖에 되지 않아 단일 분할의 결과가 우연에 의해 크게 달라질 수 있다는 점은 4장에서도 확인했습니다. 데이터 출처가 미국 Cleveland Clinic 단일 기관이라는 점도 한계입니다. 서로 다른 나라, 다른 병원, 다른 진단 기준 아래 수집된 환자군에 이 모델을 그대로 적용하면 성능이 크게 떨어질 수 있습니다.

성별 구성도 불균형합니다. 전체 303명 중 남성이 206명(68.0%), 여성이 97명(32.0%)입니다. 심장 질환은 성별에 따라 증상 양상이 다를 수 있는데, 여성 데이터가 상대적으로 적어 모델이 여성의 패턴을 충분히 학습하지 못했을 가능성이 있습니다.

8.2 모델 한계

최종 선택된 모델은 RandomForest입니다. 성능 면에서는 좋은 결과를 냈지만, 수백 개의 결정 트리를 앙상블하는 구조라 ‘왜 이 환자를 심장병으로 예측했는가’를 전문의에게 직관적으로 설명하기 어렵습니다. 만약 4장에서 CV 성능이 가장 안정적이었던 LogisticRegression을 선택했다면, 각 특성의 계수로 예측 근거를 설명하기가 훨씬 쉬웠을 것입니다. 이번 구현에서는 SHAP이나 LIME 같은 설명 가능 AI 도구를 적용하지 않아 이 한계를 보완하지 못했습니다.

분류 임계값도 기본값인 0.5를 그대로 사용했습니다. 의료 분류에서는 FN(심장병을 놓치는 것)의 임상적 비용이 FP보다 훨씬 크기 때문에, 임계값을 0.5보다 낮게 조정하면 recall을 더 높일 수 있습니다. 이 최적화는 이번 구현에서 수행하지 않았습니다.

8.3 윤리 원칙 — 보조 도구로서의 한계

이 시스템은 심장 전문의의 의사결정을 보조하는 도구이며, 모델 출력만으로 수술·처방·입원 여부를 결정하는 데 사용해서는 안 됩니다. 특히 모델이 ‘정상(0)’으로 예측하더라도 환자에게 흉통·호흡 곤란 등 증상이 있다면 반드시 추가 검사를 진행해야 합니다. 모델은 통계적 패턴을 학습한 것이지, 개별 환자의 임상 상황 전체를 이해하지 못합니다.

투명성과 감사 추적을 위해 모델 버전, 학습 일자, 사용된 데이터 범위를 로그로 기록해야 하며, 성별·연령대별 하위 집단에서 성능 격차가 생기지 않는지 정기적으로 점검해야 합니다. 특정 집단에서 FN이 집중되는 현상은 통계적 수치에는 잘 드러나지 않기 때문입니다.

8.4 1주일이 더 주어진다면

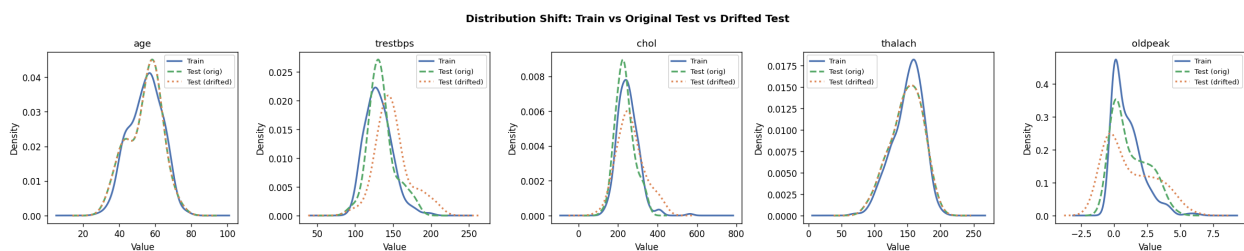
먼저 SHAP를 적용해 각 예측에서 어떤 특성이 얼마나 기여했는지 시각화하고 싶습니다. 전문의

에게 'thalach가 낮고 oldpeak가 높아 위험으로 판단했다'는 식의 근거를 보여줄 수 있으면 실제 임상에서 신뢰를 얻기 쉬울 것입니다. 그다음은 분류 임계값 최적화입니다. FN 비용과 FP 비용의 비율을 명시하고, 그에 맞는 최적 임계값을 precision-recall 곡선에서 찾는 작업을 해보고 싶습니다. 또 XGBoost나 LightGBM을 추가로 비교해 RF 선택이 실제로 최선이었는지 재검토하고, 성별·연령대별로 성능을 분리해 공정성 감사를 수행하고 싶습니다.

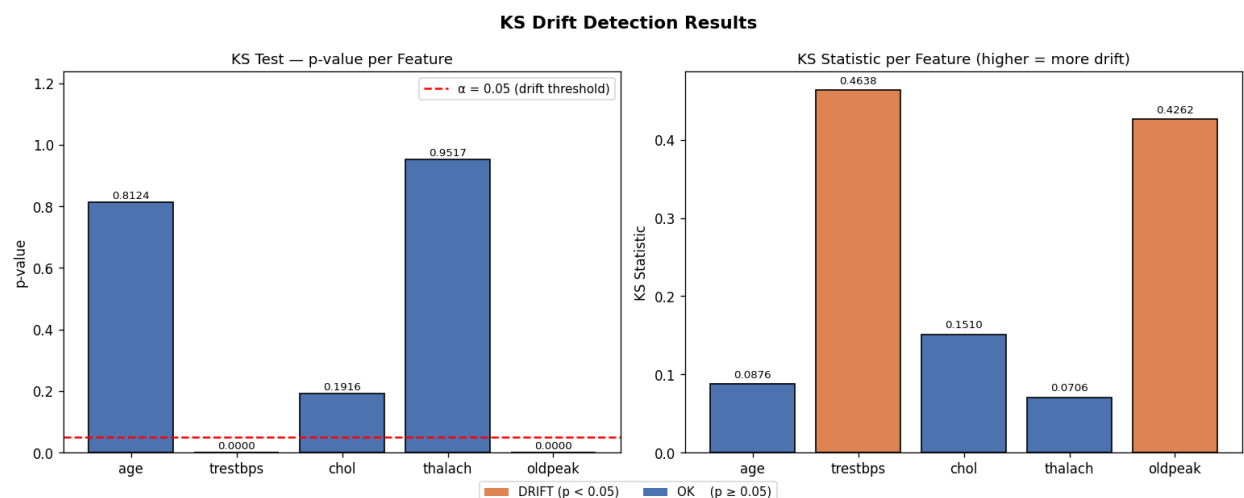
8.5 AI 사용 공개

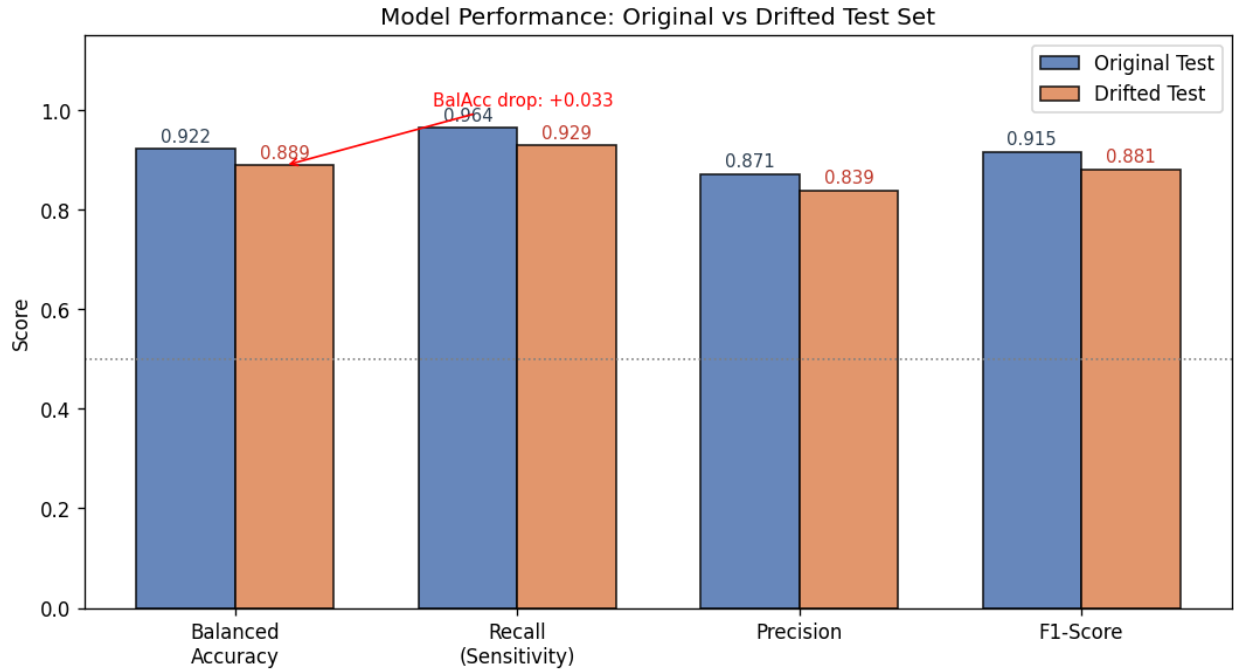
이 프로젝트에서 Claude를 코드 초안 작성, 디버깅, 보고서 문장 정리에 활용했습니다. 전처리 파이프라인 구조, MLflow 연동, Docker 설정 등 초기 구현에서 AI 도움을 많이 받았습니다. 다만 코드 전체를 직접 읽고 실행하여 각 구성 요소가 어떻게 작동하는지 이해했으며, 보고서의 분석과 해석, 모델 선택 판단, 수치 검증은 직접 수행했습니다. 제출된 코드와 보고서 내용 전체에 대해 구두 확인에 응할 수 있습니다.

부록. 모니터링 시각화

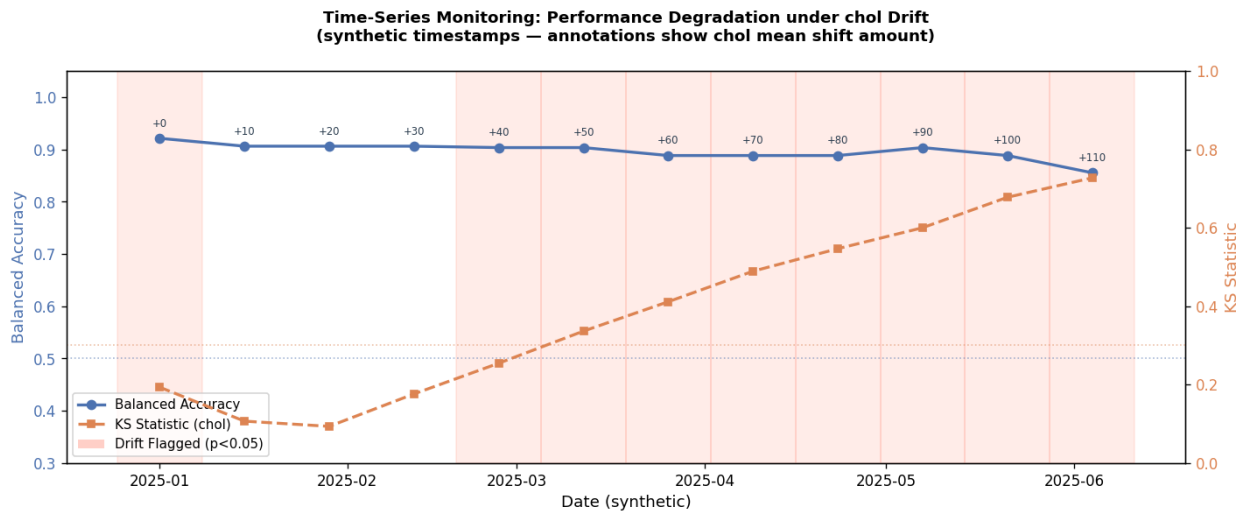


[그림 2] 분포 이동 전후 KDE





[그림 4] 원본 vs 드리프트 성능



[그림 5] 시계열 드리프트 모니터링